

Функция Эйлера

```

1 int phi (int n) {
2     int result = n;
3     for (int i=2; i*i<=n; ++i)
4         if (n % i == 0) {
5             while (n % i == 0)
6                 n /= i;
7             result -= result / i;
8         }
9     if (n > 1)
10         result -= result / n;
11     return result;
12 }

```

Расширенный алгоритм Евклида

```

1 std::tuple<int, int, int> extended_gcd(int a, int b) {
2     if (b == 0) {
3         return {a, 1, 0};
4     }
5     int g, x1, y1;
6     std::tie(g, x1, y1) = extended_gcd(b, a % b);
7     int x = y1;
8     int y = x1 - (a / b) * y1;
9     return {g, x, y};
10 }

```

Код Грея

```

1 int g (int n) {
2     return n ^ (n >> 1);
3 }
4 int rev_g (int g) {
5     int n = 0;
6     for (; g; g>>=1)
7         n ^= g;
8     return n;
9 }

```

Мосты

```

1 void dfs(int v, int height, int prev) {
2     used[v] = true;
3     h[v] = height;
4     d[v] = height;
5     for (auto [u, i] : g[v]) {
6         if (!used[u]) {
7             dfs(u, height + 1, v);
8         }
9         if (u != prev) {
10            d[v] = min(d[v], d[u]);
11            if (d[u] > h[v]) {
12                bridges.emplace_back(i);
13            }
14        }
15    }
16 }
17 void find_bridges() {
18     for (int v = 0; v < n; ++v) {
19         if (!used[v]) {
20             dfs(v, 0, -1);
21         }
22     }
23 }
24 }

```

Конденсация графа

```

1 vector<vector<int>> g;
2 vector<vector<int>> gr;
3 vector<bool> used;
4 vector<int> topsort;
5 vector<vector<int>> components;
6 map<int, int> vertex_to_comp;
7 void dfs(int v) {
8     used[v] = true;
9     for (int u : g[v]) {
10         if (!used[u]) {
11             dfs(u);
12         }
13     }
14     topsort.emplace_back(v);
15 }
16 void dfs2(int v) {
17     used[v] = true;
18     components.back().emplace_back(v);
19     vertex_to_comp[v] = components.size() - 1;
20     for (int u : gr[v]) {
21         if (!used[u]) {
22             dfs2(u);
23         }
24     }
25 }

```

```

25 }
26 int main() {
27     int n, m;
28     cin >> n >> m;
29     g.resize(n);
30     gr.resize(n);
31     used.assign(n, false);
32     for (int i = 0; i < m; ++i) {
33         int u, v;
34         cin >> u >> v;
35         u--;
36         v--;
37         g[u].emplace_back(v);
38         gr[v].emplace_back(u);
39     }
40     for (int v = 0; v < n; ++v) {
41         if (!used[v]) {
42             dfs(v);
43         }
44     }
45     fill(used.begin(), used.end(), false);
46     reverse(topsort.begin(), topsort.end());
47     for (int i = 0; i < n; ++i) {
48         if (!used[topsort[i]]) {
49             components.push_back(vector<int>());
50             dfs2(topsort[i]);
51         }
52     }
53 }

```

Форд-Беллман (NM)

```

1 void Ford_Bellman(int start) {
2     for (int i = 1; i <= n; ++i)
3         d[i] = INF;
4     d[start] = 0;
5     vector<int> cnt(n + 1, 0);
6     queue<pair<int, int>> q;
7     q.push({start, 0});
8     while (!q.empty()) {
9         auto [v, dist] = q.front();
10        q.pop();
11        if (dist > d[v])
12            continue;
13        for (auto [i, w]: g[v]) {
14            if (d[v] + w < d[i]) {
15                d[i] = d[v] + w;
16                cnt[i] = cnt[v] + 1;
17                if (cnt[i] >= n)
18                    return; //
19                q.push({i, d[i]});
20            }
21        }
22    }
23 }

```

Точки Сочленения

```

1 void dfs (int v, int p = -1) {
2     used[v] = true;
3     tin[v] = fup[v] = timer++;
4     int children = 0;
5     for (size_t i=0; i<g[v].size(); ++i) {
6         int to = g[v][i];
7         if (to == p) continue;
8         if (used[to])
9             fup[v] = min (fup[v], tin[to]);
10        else {
11            dfs (to, v);
12            fup[v] = min (fup[v], fup[to]);
13            if (fup[to] >= tin[v] && p != -1)
14                IS_CUTPOINT(v);
15            ++children;
16        }
17    }
18    if (p == -1 && children > 1)
19        IS_CUTPOINT(v);
20 }
21
22 int main() {
23     int n;
24     ...
25     timer = 0;
26     for (int i=0; i<n; ++i)
27         used[i] = false;
28     dfs (0);
29 }

```

Кун

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4 using i64 = long long;

```

```

5
6 vector<vector<int>>> g;
7 vector<int> mt;
8 vector<bool> used;
9
10 bool dfs(int v) {
11     if (used[v]) {
12         return false;
13     }
14     used[v] = true;
15     for (int u : g[v]) {
16         if (mt[u] == -1 || dfs(mt[u])) {
17             mt[u] = v;
18             return true;
19         }
20     }
21     return false;
22 }
23
24
25 int main() {
26     ios_base::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     int n, m;
30     cin >> n >> m;
31
32     g.resize(n);
33     mt.resize(m, -1);
34
35     for (int i = 0; i < n; ++i) {
36         int v;
37         cin >> v;
38         while (v != 0) {
39             v--;
40             g[i].emplace_back(v);
41             cin >> v;
42         }
43     }
44
45     int cnt = 0;
46
47     for (int i = 0; i < n; ++i) {
48         used.assign(n, false);
49         if (dfs(i)) {
50             cnt++;
51         }
52     }
53
54     cout << cnt << '\n';
55 }

```

Максимальное паросочетание (Диница) $O(m\sqrt{n})$

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4 using i64 = long long;
5
6 const int INF = 1e9;
7
8 struct Edge {
9     int to;
10    int rev;
11    int flow;
12    int capacity;
13 };
14
15 vector<Edge> edges;
16 vector<vector<int>>> g;
17 vector<int> dist;
18 vector<int> current_edge;
19 int S, T;
20 int total_flow = 0;
21
22 int dfs(int v, int min_capacity) {
23     if (v == T) {
24         return min_capacity;
25     }
26
27     for (; current_edge[v] < g[v].size(); ++current_edge[v]) {
28         int i = g[v][current_edge[v]];
29         Edge& e = edges[i];
30
31         if (dist[e.to] == dist[v] + 1 && e.capacity - e.flow > 0) {
32             int flow = dfs(e.to, min(min_capacity, e.capacity - e.flow));
33             if (flow > 0) {
34                 e.flow += flow;
35                 edges[e.rev].flow -= flow;
36                 return flow;
37             }
38         }
39     }
40     return 0;
41 }

```

```

42
43 bool bfs() {
44     fill(dist.begin(), dist.end(), -1);
45     dist[S] = 0;
46     queue<int> q;
47     q.push(S);
48
49     while (!q.empty()) {
50         int v = q.front();
51         q.pop();
52
53         for (int i : g[v]) {
54             Edge& e = edges[i];
55             if (e.capacity - e.flow > 0 && dist[e.to] == -1) {
56                 dist[e.to] = dist[v] + 1;
57                 q.push(e.to);
58             }
59         }
60     }
61
62     return dist[T] != -1;
63 }
64
65 void max_flow() {
66     while (bfs()) {
67         fill(current_edge.begin(), current_edge.end(), 0);
68         while (true) {
69             int flow = dfs(S, INF);
70             if (flow <= 0) {
71                 break;
72             }
73             total_flow += flow;
74         }
75     }
76 }
77
78 void add_edge(int from, int to, int capacity) {
79     int forward_edge = edges.size();
80     int backward_edge = forward_edge + 1;
81
82     g[from].push_back(forward_edge);
83     g[to].push_back(backward_edge);
84
85     edges.push_back({to, backward_edge, 0, capacity});
86     edges.push_back({from, forward_edge, 0, 0});
87 }
88
89 int main() {
90     ios_base::sync_with_stdio(false);
91     cin.tie(nullptr);
92
93     int n, m;
94     cin >> n >> m;
95
96     g.resize(n);
97     current_edge.resize(n);
98     dist.resize(n);
99
100     for (int i = 0; i < m; ++i) {
101         int u, v, c;
102         cin >> u >> v;
103         --u; --v;
104
105         add_edge(u, v, 1);
106         add_edge(v, u, 1);
107     }
108
109     S = 0;
110     T = n - 1;
111     max_flow();
112
113     cout << total_flow << '\n';
114
115     for (int i = 0; i < edges.size(); i += 4) {
116         int flow = edges[i].flow;
117     }
118
119     return 0;
120 }

```

Прима

```

1 int n;
2 vector < vector<int> > g;
3 const int INF = 1000000000;
4 vector<bool> used (n);
5 vector<int> min_e (n, INF), sel_e (n, -1);
6 min_e[0] = 0;
7 for (int i=0; i<n; ++i) {
8     int v = -1;
9     for (int j=0; j<n; ++j)
10         if (!used[j] && (v == -1 || min_e[j] < min_e[v]))
11             v = j;
12     if (min_e[v] == INF) {
13         cout << "No MST!";
14         exit(0);

```

```

15     }
16     used[v] = true;
17     if (sel_e[v] != -1)
18         cout << v << " " << sel_e[v] << endl;
19     for (int to=0; to<n; ++to)
20         if (g[v][to] < min_e[to]) {
21             min_e[to] = g[v][to];
22             sel_e[to] = v;
23         }
24 }

```

Отрицательный цикл

```

1 struct edge {
2     int a, b, cost;
3 };
4 int n, m;
5 vector<edge> e;
6 const int INF = 1000000000;
7 void solve() {
8     vector<int> d (n);
9     vector<int> p (n, -1);
10    int x;
11    for (int i=0; i<n; ++i) {
12        x = -1;
13        for (int j=0; j<m; ++j)
14            if (d[e[j].b] > d[e[j].a] + e[j].cost) {
15                d[e[j].b] = max(-INF, d[e[j].a] + e[j].cost);
16                p[e[j].b] = e[j].a;
17                x = e[j].b;
18            }
19    }
20    if (x == -1)
21        cout << "No negative cycle found.";
22    else {
23        int y = x;
24        for (int i=0; i<n; ++i)
25            y = p[y];
26        vector<int> path;
27        for (int cur=y; ; cur=p[cur]) {
28            path.push_back (cur);
29            if (cur == y && path.size() > 1) break;
30        }
31        reverse (path.begin(), path.end());
32        cout << "Negative cycle: ";
33        for (size_t i=0; i<path.size(); ++i)
34            cout << path[i] << ' ';
35    }
36 }

```

LCA

```

1 const int N = 1000010, LOG = 21;
2 vector<int> g[N];
3 int tin[N], tout[N], timer = 0;
4 int up[LOG][N];
5
6 void dfs(int v, int p) {
7     tin[v] = ++timer;
8     up[0][v] = (p == -1 ? v : p);
9     for (int i = 1; i < LOG; ++i) {
10         up[i][v] = up[i-1][up[i-1][v]];
11     }
12     for (int i : g[v]) {
13         if (i == p) continue;
14         dfs(i, v);
15     }
16     tout[v] = timer;
17 }
18
19 int upper(int a, int b) {
20     return tin[a] <= tin[b] && tout[a] >= tout[b];
21 }
22
23 int LCA(int a, int b) {
24     if (upper(a, b)) return a;
25     if (upper(b, a)) return b;
26     for (int i = LOG - 1; i >= 0; --i) {
27         if (!upper(up[i][a], b))
28             a = up[i][a];
29     }
30     return up[0][a];
31 }

```

Максимальный поток (Масштабирование, MM logC)

```

1 struct edge {

```

```

2     ll to, c, f;
3 };
4
5 const int N = 1010;
6 ll cur_flow, s, t;
7 ll d[N], ptr[N];
8 vector<int> g[N];
9 vector<edge> e;
10
11 void init() {
12     for (int i = 1; i < N; ++i) {
13         d[i] = inf;
14         ptr[i] = 0;
15     }
16 }
17
18 void add(ll a, ll b, ll c) {
19     g[a].push_back(sz(e));
20     e.push_back({b, c, 0});
21     g[b].push_back(sz(e));
22     e.push_back({a, 0, 0});
23 }
24
25 int bfs() {
26     queue<ll> q;
27     q.push(s);
28     d[s] = 0;
29     while (!q.empty()) {
30         ll v = q.front();
31         q.pop();
32         for (int i: g[v]) {
33             if (e[i].c - e[i].f >= cur_flow && d[e[i].to] == inf) {
34                 d[e[i].to] = d[v] + 1;
35                 q.push(e[i].to);
36             }
37         }
38     }
39     return (d[t] != inf);
40 }
41
42 int dfs(ll v) {
43     if (v == t)
44         return 1;
45     for (ll i = ptr[v]; i < sz(g[v]); ++i) {
46         int num = g[v][i];
47         if (d[e[num].to] == d[v] + 1 && e[num].c - e[num].f >= cur_flow) {
48             int x = dfs(e[num].to);
49             if (x) {
50                 e[num].f += cur_flow;
51                 e[num ^ 1].f -= cur_flow;
52                 return 1;
53             }
54         }
55         ++ptr[v];
56     }
57     return 0;
58 }
59
60 ll max_flow() {
61     ll res = 0;
62     for (ll power = 32; power >= 0; --power) {
63         cur_flow = (1ll << power);
64         init();
65         while (bfs()) {
66             int x = dfs(s);
67             while (x) {
68                 res += cur_flow;
69                 x = dfs(s);
70             }
71             init();
72         }
73     }
74     return res;
75 }

```

RMQ

```

1 #include <cstring>
2 #include <iostream>
3
4 using namespace std;
5
6 using i32 = int;
7 using i64 = long long;
8
9 const int maxn = 1e5 + 1;
10 const int logn = 17;
11
12 int a[maxn], mn[logn][maxn];
13
14 int rmq(int l, int r) {
15     int t = __lg(r - l + 1);
16     return min(mn[t][l], mn[t][r + 1 - (1 << t)]);
17 }
18
19 int main() {

```

```

20     int n, m;
21
22     memcpy(mn[0], a, sizeof a);
23     for (int l = 0; l < logn - 1; ++l) {
24         for (int i = 0; i + (2 << l) <= n; ++i) {
25             mn[l + 1][i] = min(mn[l][i], mn[l][i + (1 << l)]);
26         }
27     }
28
29     int ansi = rmq(min(u, v) - 1, max(u, v) - 1);
30
31     for (int i = 1; i < m; ++i) {
32         ansi = rmq(min(u, v) - 1, max(u, v) - 1);
33     }
34
35     return 0;
36 }

```

Максимальный поток (NNN)

```

1  const int INF = 1000*1000*1000;
2  int main() {
3      int n;
4      vector < vector<int> > c (n, vector<int> (n));
5      int s, t;
6      // cin n c t s
7      vector<int> e (n);
8      vector<int> h (n);
9      h[s] = n-1;
10     vector < vector<int> > f (n, vector<int> (n));
11     for (int i=0; i<n; ++i) {
12         f[s][i] = c[s][i];
13         f[i][s] = -f[s][i];
14         e[i] = c[s][i];
15     }
16     vector<int> maxh (n);
17     int sz = 0;
18     for (;;) {
19         if (!sz)
20             for (int i=0; i<n; ++i)
21                 if (i != s && i != t && e[i] > 0) {
22                     if (sz && h[i] > h[maxh[0]])
23                         sz = 0;
24                     if (!sz || h[i] == h[maxh[0]])
25                         maxh[sz++] = i;
26                 }
27         if (!sz) break;
28         while (sz) {
29             int i = maxh[sz-1];
30             bool pushed = false;
31             for (int j=0; j<n && e[i]; ++j)
32                 if (c[i][j]-f[i][j] > 0 && h[i] == h[j]+1) {
33                     pushed = true;
34                     int addf = min (c[i][j]-f[i][j], e[i]);
35                     f[i][j] += addf, f[j][i] -= addf;
36                     e[i] -= addf, e[j] += addf;
37                     if (e[i] == 0) --sz;
38                 }
39             if (!pushed) {
40                 h[i] = INF;
41                 for (int j=0; j<n; ++j)
42                     if (c[i][j]-f[i][j] > 0 && h[j]+1 < h[i])
43                         h[i] = h[j]+1;
44                 if (h[i] > h[maxh[0]]) {
45                     sz = 0;
46                     break;
47                 }
48             }
49         }
50     }
51 }

```

Стрессы

```

1  import os
2  ti = 0 # seed number
3  while True:
4      os.system(f"python3 gen.py {ti} > .in")
5      return_code = os.system("./correct < .in > .outc && ./incorrect < .in > .outi")
6      if return_code:
7          print("re:", ti)
8          exit()
9      if open(".outi").read() != open(".outc").read():
10         print("wa:", ti)
11         exit()
12     ti+=1
13     if ti % 25 == 0:
14         print(ti)

```

```

1  import random
2  import sys
3  seed = int(sys.argv[1])
4  random.seed(seed)

```

Санитайзеры

```

1 [G++ fast run]
2 g++ -std=c++20 -O2 -pipe -Wall -Wextra -Wshadow -Wconversion -Wno-unused-result -DLOCAL -static -s
   main.cpp -o main.exe && main.exe
3 [G++ with sanitizers (ASan+UBSan)]
4 g++ -std=c++20 -O1 -g -Wall -Wextra -Wshadow -fno-omit-frame-pointer -fsanitize=address,undefined
   main.cpp -o main_asan.exe && main_asan.exe
5 [MSVC fast run]
6 cl /std:c++20 /O2 /EHsc /W4 /DLOCAL /nologo main.cpp && main.exe
7 [MSVC with AddressSanitizer]
8 cl /std:c++20 /O2 /EHsc /W4 /fsanitize=address /Zi /nologo main.cpp && main.exe

```

Проверка двух отрезков на пересечение

```

1 struct pt {
2     int x, y;
3 };
4 inline int area (pt a, pt b, pt c) {
5     return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
6 }
7 inline bool intersect_1 (int a, int b, int c, int d) {
8     if (a > b) swap (a, b);
9     if (c > d) swap (c, d);
10    return max(a,c) <= min(b,d);
11 }
12 bool intersect (pt a, pt b, pt c, pt d) {
13     return intersect_1 (a.x, b.x, c.x, d.x)
14         && intersect_1 (a.y, b.y, c.y, d.y)
15         && area(a,b,c) * area(a,b,d) <= 0
16         && area(c,d,a) * area(c,d,b) <= 0;
17 }

```

Конвекс Хулл

```

1 vector<point> convex_hull(vector<point> &p) {
2     int mn = 0;
3     for (int i = 1; i < sz(p); ++i) {
4         if (p[mn].y > p[i].y || p[mn].y == p[i].y && p[mn].x > p[i].x)
5             mn = i;
6     }
7     point p0 = p[mn];
8     p.erase(find(p.begin(), p.end(), p0));
9     for (auto &i: p)
10        i = i - p0;
11    point add = p0;
12    p0 = {0, 0};
13    sort(p.begin(), p.end(), [](point a, point b) {
14        return (a ^ b) > 0 || (a ^ b) == 0 && a.len() > b.len();
15    });
16    p.push_back(p0);
17    vector<point> hull;
18    for (auto &i: p) {
19        while (sz(hull) > 1 && ((hull.back() - hull[sz(hull) - 2]) ^ (i - hull.back())) <= 0) {
20            hull.pop_back();
21        }
22        hull.push_back(i);
23    }
24    for (auto &i: hull)
25        i = i + add;
26    return hull;
27 }

```

Проверка точки на принадлежность выпуклому многоугольнику

```

1 struct pt {int x, y;};
2 struct ang {int a, b;};
3 bool operator < (const ang &p, const ang &q) {
4     if (p.b == 0 && q.b == 0)
5         return p.a < q.a;
6     return p.a * 1ll * q.b < p.b * 1ll * q.a;
7 }
8 long long sq (pt &a, pt &b, pt &c) {return a.x*1ll*(b.y-c.y) + b.x*1ll*(c.y-a.y) + c.x*1ll*(a.y-b.y);}
9 int main() {
10    int n;
11    cin >> n;
12    vector<pt> p (n);
13    int zero_id = 0;
14    for (int i=0; i<n; ++i) {
15        scanf ("%d%d", &p[i].x, &p[i].y);
16        if (p[i].x < p[zero_id].x || p[i].x == p[zero_id].x && p[i].y < p[zero_id].y)
17            zero_id = i;
18    }
19    pt zero = p[zero_id];
20    rotate (p.begin(), p.begin()+zero_id, p.end());
21    p.erase (p.begin());
22    --n;
23    vector<ang> a (n);
24    for (int i=0; i<n; ++i) {
25        a[i].a = p[i].y - zero.y;
26        a[i].b = p[i].x - zero.x;
27        if (a[i].a == 0)
28            a[i].b = a[i].b < 0 ? -1 : 1;

```

```

29     }
30     for (;;) {
31         pt q;
32         bool in = false;
33         if (q.x >= zero.x)
34             if (q.x == zero.x && q.y == zero.y)
35                 in = true;
36             else {
37                 ang my = { q.y-zero.y, q.x-zero.x };
38                 if (my.a == 0)
39                     my.b = my.b < 0 ? -1 : 1;
40                 vector<ang>::iterator it = upper_bound (a.begin(), a.end(), my);
41                 if (it == a.end() && my.a == a[n-1].a && my.b == a[n-1].b)
42                     it = a.end()-1;
43                 if (it != a.end() && it != a.begin()) {
44                     int p1 = int (it - a.begin());
45                     if (sq (p[p1], p[p1-1], q) <= 0)
46                         in = true;
47                 }
48             }
49         puts (in ? "INSIDE" : "OUTSIDE");
50     }
51 }

```

Z - функция

```

1 vector<int> z_function (string s) {
2     int n = (int) s.length();
3     vector<int> z (n);
4     for (int i=1, l=0, r=0; i<n; ++i) {
5         if (i <= r)
6             z[i] = min (r-i+1, z[i-1]);
7         while (i+z[i] < n && s[z[i]] == s[i+z[i]])
8             ++z[i];
9         if (i+z[i]-1 > r)
10             l = i, r = i+z[i]-1;
11     }
12     return z;
13 }

```

Префикс-функция

```

1 vector<int> prefix_function (string s) {
2     int n = (int) s.length();
3     vector<int> pi (n);
4     for (int i=1; i<n; ++i) {
5         int j = pi[i-1];
6         while (j > 0 && s[i] != s[j])
7             j = pi[j-1];
8         if (s[i] == s[j]) ++j;
9         pi[i] = j;
10    }
11    return pi;
12 }

```

Фенвик

```

1 vector<int> t;
2 int n;
3 void init (int nn)
4 {
5     n = nn;
6     t.assign (n, 0);
7 }
8 int sum (int r)
9 {
10    int result = 0;
11    for (; r >= 0; r = (r & (r+1)) - 1)
12        result += t[r];
13    return result;
14 }
15 void inc (int i, int delta)
16 {
17    for (; i < n; i = (i | (i+1)))
18        t[i] += delta;
19 }
20 int sum (int l, int r)
21 {
22    return sum (r) - sum (l-1);
23 }
24 void init (vector<int> a)
25 {
26    init ((int) a.size());
27    for (unsigned i = 0; i < a.size(); i++)
28        inc (i, a[i]);
29 }

```

CHM

```

1 #include <numeric>
2 struct dsu {
3     int n;
4     vector<int> p;

```

```

5     vector<int> r;
6
7     dsu(int n) : n(n) {
8         p.resize(n);
9         iota(p.begin(), p.end(), 0);
10        r.assign(n, 1);
11    }
12
13    int get(int a) {
14        if (p[a] != a) {
15            p[a] = get(p[a]);
16        }
17        return p[a];
18    }
19
20    void join(int a, int b) {
21        a = get(a);
22        b = get(b);
23
24        if (a == b) {
25            return;
26        }
27
28        if (r[a] == r[b]) {
29            r[a]++;
30        }
31
32        if (r[a] > r[b]) {
33            p[b] = a;
34        } else {
35            p[a] = b;
36        }
37    }
38 };

```

Наибольшая возрастающая подпоследовательность

```

1 vector<int> d(MAXN);
2 d[0] = -INF;
3 for (int i=1; i<=n; ++i)
4     d[i] = INF;
5 for (int i=0; i<n; i++) {
6     int j = int(upper_bound(d.begin(), d.end(), a[i]) - d.begin());
7     if (d[j-1] < a[i] && a[i] < d[j])
8         d[j] = a[i];
9 }

```

Путь эйлера

```

1 vector<ll> res;
2 void eiler(ll v, vector<vector<pair<ll, ll>>>& gr, vector<char>& used) {
3     for (ll i = 0; i < gr[v].size(); ++i) {
4         ll to = gr[v][i].first;
5         ll ind = gr[v][i].second;
6         if (used[ind])
7             continue;
8         used[ind] = 1;
9         eiler(to, gr, used);
10    }
11    res.push_back(v);
12 }

```

Хэши

```

1 i64 MD1 = 1e9 + 7;
2 i64 MD2 = 1e9 + 9;
3
4 struct Mint {
5     i64 x1, x2;
6
7     Mint (i64 x = 0) : x1(((x % MD1) + MD1) % MD1), x2(((x % MD2) + MD2) % MD2) {}
8
9     Mint (i64 x1, i64 x2) : x1(((x1 % MD1) + MD1) % MD1), x2(((x2 % MD2) + MD2) % MD2) {}
10
11     friend Mint operator+(Mint l, Mint r) {
12         return Mint(l.x1 + r.x1, l.x2 + r.x2);
13     }
14     friend Mint operator-(Mint l, Mint r) {
15         return Mint(l.x1 - r.x1, l.x2 - r.x2);
16     }
17     friend Mint operator* (Mint l, Mint r) {
18         return Mint(l.x1 * r.x1, l.x2 * r.x2);
19     }
20     friend bool operator==(Mint l, Mint r) {
21         return l.x1 == r.x1 && l.x2 == r.x2;
22     }
23     friend bool operator!=(Mint l, Mint r) {
24         return l.x1 != r.x1 || l.x2 != r.x2;
25     }
26     friend bool operator<(Mint l, Mint r) {
27         return l.x1 < r.x1 || (l.x1 == r.x1 && l.x2 < r.x2);
28     }
29     void operator+=(Mint r) {
30         *this = *this + r;

```

```

31     }
32     void operator--(Mint r) {
33         *this = *this - r;
34     }
35     void operator*=(Mint r) {
36         *this = *this * r;
37     }
38 };
39
40 Mint p = 179;
41 vector<Mint> degp;
42
43 int maxnum = 100000;
44
45 vector<Mint> h;
46
47 int n;
48 Mint hash_substring(int l, int r) {
49     return (h[r + 1] - h[l]) * degp[n - 1];
50 }
51
52 int main() {
53     degp.resize(maxnum + 1);
54     degp[0] = 1;
55     for (int i = 1; i <= maxnum; ++i) {
56         degp[i] = degp[i - 1] * p;
57     }
58
59     string s;
60     cin >> s;
61
62     n = s.size();
63
64     h.resize(n + 1);
65     h[0] = 0;
66     for (int i = 1; i <= n; ++i) {
67         h[i] = h[i - 1] + degp[i - 1] * (s[i - 1] - 'a' + 1);
68     }
69 }

```

«Деление» по модулю

```

1  const int mod = 1e9 + 7;
2  int binpow(int a, int n) {
3      int res = 1;
4      while (n != 0) {
5          if (n & 1)
6              res = res * a % mod;
7          a = a * a % mod;
8          n >>= 1;
9      }
10     return res;
11 }
12
13 int inv(int x) {
14     return binpow(x, mod - 2);
15 }

```

Метод отжига

```

1  const int n = 100;
2  const int k = 1000;
3  int f(vector<int> &p) {
4      int s = 0;
5      for (int i = 0; i < n; i++) {
6          int d = 1;
7          for (int j = 0; j < i; j++)
8              if (abs(i - j) == abs(p[i] - p[j]))
9                  d = 0;
10         s += d;
11     }
12     return s;
13 }
14 double rnd() { return double(rand()) / RAND_MAX; }
15 int main() {
16     vector<int> v(n);
17     iota(v.begin(), v.end(), 0);
18     mt19937 gen(random_device{}());
19     shuffle(v.begin(), v.end(), gen);
20     int ans = f(v);
21
22     double t = 1;
23     // USE clock() < CLOCKS_PER_SEC * 0.95 from #include <ctime>
24     for (int i = 0; i < k && ans < n; i++) {
25         t *= 0.99;
26         vector<int> u = v;
27         swap(u[rand() % n], u[rand() % n]);
28         int val = f(u);
29         if (val > ans || rnd() < exp((val - ans) / t)) {
30             v = u;
31             ans = val;
32         }
33     }
34
35     for (int x : v)

```

```

36         cout << x + 1 << " ";
37
38     return 0;
39 }

```

Линейные уравнения

```

1 typedef vector< vector<double> > matrix;
2
3 vector<double> gauss(matrix a) {
4     int n = (int) a.size();
5     for (int i = 0; i < n; i++) {
6         int best = i;
7         for (int j = i + 1; j < n; j++)
8             if (abs(a[j][i]) > abs(a[best][i]))
9                 best = j;
10        swap(a[best], a[i]);
11        for (int j = n; j >= i; j--)
12            a[i][j] /= a[i][i];
13        for (int j = 0; j < n; j++)
14            if (j != i)
15                for (int k = n; k >= i; k--)
16                    a[j][k] -= a[i][k] * a[j][i];
17    }
18    vector<double> ans(n);
19    for (int i = 0; i < n; i++)
20        ans[i] = a[i][n];
21    return ans;
22 }

```

Ordered Set

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4 #define ordered_set tree<int, null_type, std::less<int>, rb_tree_tag,
5     tree_order_statistics_node_update>
6
7 ordered_set o_set;
8
9 o_set.insert(2);
10
11 // Finding the second smallest element
12 *(o_set.find_by_order(1))
13
14 // Finding the number of elements strictly less than k=4
15 o_set.order_of_key(4)
16
17 // Deleting 2 from the set if it exists
18 if (o_set.find(2) != o_set.end()) o_set.erase(o_set.find(2));

```

Решето Эратосфена

```

1 vector<bool> sieve(int n) {
2     vector<bool> is_prime(n + 1, true);
3     for (int i = 2; i <= n; i++)
4         if (is_prime[i])
5             for (int j = 2 * i; j <= n; j += i)
6                 is_prime[j] = false;
7     return is_prime;
8 }

```

Решето Эратосфена за линию

```

1 const int n = 1e6;
2
3 int d[n + 1];
4 vector<int> p;
5
6 for (int k = 2; k <= n; k++) {
7     if (d[k] == 0) {
8         d[k] = k;
9         p.push_back(k);
10    }
11    for (int x : p) {
12        if (x > d[k] || x * k > n)
13            break;
14        d[k * x] = x;
15    }
16 }

```

Дейкстра mlogn

```

1 vector<int> dijkstra(int s) {
2     vector<int> d(n, inf);
3     d[s] = 0;
4     using Pair = pair<int, int>;
5     priority_queue<Pair, vector<Pair>, greater<Pair>> q;
6     q.push({0, s});
7     while (!q.empty()) {
8         auto [cur_d, v] = q.top();
9         q.pop();

```

```

10     if (cur_d > d[v])
11         continue;
12     for (auto [u, w] : g[v]) {
13         if (d[u] > d[v] + w) {
14             d[u] = d[v] + w;
15             q.push({d[u], u});
16         }
17     }
18 }
19 return d;
20 }

```

Рюкзак

```

1 for i in range(1, n + 1):
2     for j in range(0, W + 1):
3         dp[i][j] = dp[i - 1][j]
4         if a[i] <= j:
5             dp[i][j] = max(dp[i][j], dp[i - 1][j - a[i]] + c[i])

```

Файлы

```

1 #include <fstream>
2 std::ifstream fin("input.txt");
3 std::ofstream fout("output.txt");
4
5 #include <cstdio>
6 freopen("input.txt", "r", stdin);
7 freopen("output.txt", "w", stdout);

```

ДО (сумма на отрезке, изменение в точке)

```

1 struct SegTree {
2     int size;
3     vector<int> tree;
4
5     void init(int n) {
6         size = 1;
7         while (size < n) size *= 2;
8         tree.assign(2 * size - 1, 0);
9     }
10
11     void build(vector<int> &a, int x, int lx, int rx) {
12         if (rx - lx == 1) {
13             if (lx < a.size()) tree[x] = a[lx];
14             return;
15         }
16         int m = (lx + rx) / 2;
17         build(a, 2 * x + 1, lx, m);
18         build(a, 2 * x + 2, m, rx);
19         tree[x] = tree[2 * x + 1] + tree[2 * x + 2];
20     }
21
22     void build(vector<int> &a) {
23         init(a.size());
24         build(a, 0, 0, size);
25     }
26
27     void set(int i, int v, int x, int lx, int rx) {
28         if (rx - lx == 1) {
29             tree[x] = v;
30             return;
31         }
32         int m = (lx + rx) / 2;
33         if (i < m) set(i, v, 2 * x + 1, lx, m);
34         else set(i, v, 2 * x + 2, m, rx);
35         tree[x] = tree[2 * x + 1] + tree[2 * x + 2];
36     }
37
38     void set(int i, int v) {
39         set(i, v, 0, 0, size);
40     }
41
42     int sum(int l, int r, int x, int lx, int rx) {
43         if (lx >= r || rx <= l) return 0;
44         if (lx >= l && rx <= r) return tree[x];
45         int m = (lx + rx) / 2;
46         return sum(l, r, 2 * x + 1, lx, m) + sum(l, r, 2 * x + 2, m, rx);
47     }
48
49     int sum(int l, int r) {
50         return sum(l, r, 0, 0, size);
51     }
52 };

```

ДО (kth единица)

```

1 int kth(int k, int x, int lx, int rx) {
2     if (rx - lx == 1) {
3         return x - size + 1;
4     }
5     int m = (lx + rx) / 2;
6     if (tree[2 * x + 1] <= k) {

```

```

7         return kth(k - tree[2 * x + 1], 2 * x + 2, m, rx);
8     } else {
9         return kth(k, 2 * x + 1, lx, m);
10    }
11 }
12
13 int kth(int k) {
14     return kth(k, 0, 0, size);
15 }

```

ДО (прибавление и сумма на отрезке)

```

1 struct SegTree {
2     struct node {
3         ll add;
4         ll sum;
5     };
6
7     vector<node> tree;
8     int size = 1;
9     const ll null = 0;
10
11     void init(int n) {
12         while (size < n) size *= 2;
13     }
14
15     void build(int n) {
16         init(n);
17         tree.assign(2 * size - 1, {0, 0});
18     }
19
20     void propagate(int x, int lx, int rx) {
21         if (rx - lx == 1) return;
22
23         tree[2 * x + 1].add += tree[x].add / 2;
24         tree[2 * x + 1].sum += tree[x].add / 2;
25         tree[2 * x + 2].add += tree[x].add / 2;
26         tree[2 * x + 2].sum += tree[x].add / 2;
27
28         tree[x].add = 0;
29     }
30
31     void modify(int l, int r, ll v, int x, int lx, int rx) {
32         propagate(x, lx, rx);
33
34         if (rx <= l || lx >= r) return;
35
36         if (lx >= l && rx <= r) {
37             tree[x].add += v * (rx - lx);
38             tree[x].sum += v * (rx - lx);
39             return;
40         }
41
42         int m = (lx + rx) / 2;
43         modify(l, r, v, 2 * x + 1, lx, m);
44         modify(l, r, v, 2 * x + 2, m, rx);
45         tree[x].sum = tree[2 * x + 1].sum + tree[2 * x + 2].sum;
46     }
47
48     void modify(int l, int r, ll v) {
49         modify(l, r, v, 0, 0, size);
50     }
51
52     ll get(int l, int r, int x, int lx, int rx) {
53         propagate(x, lx, rx);
54
55         if (rx <= l || lx >= r) return 0;
56         if (lx >= l && rx <= r) return tree[x].sum;
57
58         int m = (lx + rx) / 2;
59         return get(l, r, 2 * x + 1, lx, m) + get(l, r, 2 * x + 2, m, rx);
60     }
61
62     ll get(int l, int r) {
63         return get(l, r, 0, 0, size);
64     }
65 };

```

Геометрия

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4 #define sz(v) (int)(v).size()
5 const double EPS = 1e-9;
6 const double PI = 3.141592653;
7
8 int eq(double a, double b) {
9     return abs(a - b) < EPS;
10 }
11
12 struct Point {
13     double x, y;
14
15     Point(double x = 0, double y = 0): x(x), y(y) {}

```

```

16
17     Point operator + (Point other) const {
18         return {x+other.x,y+other.y};
19     }
20
21     Point operator - (Point other) const {
22         return {x-other.x,y-other.y};
23     }
24
25     Point operator * (double k) const {
26         return {x * k, y * k};
27     }
28
29     Point operator / (double k) const {
30         return {x / k, y / k};
31     }
32
33     double operator ^ (Point other) const {
34         return x*other.y - y*other.x;
35     }
36
37     double operator % (Point other) const {
38         return x*other.x + y*other.y;
39     }
40
41     bool operator == (Point other) const {
42         return eq(x, other.x) && eq(y, other.y);
43     }
44
45     bool operator != (Point other) const {
46         return !eq(x, other.x) || !eq(y, other.y);
47     }
48
49     bool operator < (Point other) const {
50         return x<other.x-EPS || eq(x, other.x) && y<other.y;
51     }
52     double len() const {
53         return sqrt(x*x + y*y);
54     }
55 };
56 istream& operator >> (istream& is, Point &p) {
57     is >> p.x >> p.y;
58     return is;
59 }
60 ostream& operator << (ostream& os, Point &p) {
61     os << p.x << ' ' << p.y;
62     return os;
63 }
64 struct line{
65     double a, b, c;
66 };
67 void out(Point p) {
68     cout << p.x << ' ' << p.y << '\n';
69 }
70 line get_line(Point p1, Point p2) {
71     return { p2.y-p1.y, p1.x-p2.x, -(p1^p2) };
72 }
73 int parallel(line l1, line l2) {
74     return eq(l1.a, 0) && eq(l2.a, 0) && -l1.c / l1.b != -l2.c / l2.b ||
75         eq(l1.b, 0) && eq(l2.b, 0) && -l1.c / l1.a != -l2.c / l2.a ||
76         eq(l1.a / l2.a, l1.b / l2.b) && (!(eq(l1.c, 0) && eq(l2.c, 0)) || !eq(l1.a / l2.a, l1.c /
77         l2.c));
78 }
79 int Point_on_line(Point p, Point a, Point b) {
80     return eq((b-a)^(p-a), 0);
81 }
82 int Point_on_line2(Point p, line l) {
83     return eq(l.a*p.x + l.b*p.y + l.c, 0);
84 }
85 int Point_on_ray(Point p, Point a, Point b) {
86     return Point_on_line(p, a, b) && ((b-a)%(p-a) >= 0);
87 }
88 int Point_on_segment(Point p, Point a, Point b) {
89     return Point_on_line(p, a, b) && ((a-p)%(b-p) <= 0);
90 }
91 int Point_in_corner(Point p, Point a, Point o, Point b) {
92     Point v1 = a-o, v2 = b-o, v3 = p-o;
93     return (v1^v2)*(v1^v3) >= 0 && (v2^v1)*(v2^v3) >= 0;
94 }
95 double dist_Point_Point(Point p1, Point p2) {
96     return (p1-p2).len();
97 }
98 double dist_Point_line(Point p, Point a, Point b) {
99     return abs((p-a)^(p-b)) / (a-b).len();
100 }
101 double dist_Point_line2(Point p, line l) {
102     return abs(l.a*p.x + l.b*p.y + l.c) / sqrt(l.a*l.a+l.b*l.b);
103 }
104 double dist_Point_ray(Point p, Point a, Point b)
105 {
106     return ((b-a)%(p-a) >= 0? dist_Point_line(p, a, b) : dist_Point_Point(a, p));
107 }
108 double dist_Point_segment(Point p, Point a, Point b)
109 {
110     return ((b-a)%(p-a) >= 0 && (a-b)%(p-b) >= 0? dist_Point_line(p, a, b) : min(dist_Point_Point(a,
111     p),dist_Point_Point(b, p)));

```

```

110 }
111 double dist_line_line2(line l1, line l2)
112 {
113     Point p;
114     if (l1.b == 0) p = { -l1.c / l1.a, 0 };
115     else p = { 0, -l1.c / l1.b };
116     return dist_Point_line2(p, l2);
117 }
118 double dist_line_line(Point a, Point b, Point c, Point d)
119 {
120     line ab = get_line(a, b);
121     line cd = get_line(c, d);
122     if (!parallel(ab, cd))
123         return 0;
124     return dist_line_line2(ab, cd);
125 }
126 Point intersection_line_line(line l1, line l2)
127 {
128     return {(l2.c*l1.b - l1.c*l2.b) / (l1.a*l2.b - l2.a*l1.b),
129            (l2.c*l1.a - l1.c*l2.a) / (l2.a*l1.b - l1.a*l2.b)};
130 }
131 set <Point> intersection_seg_seg(Point a, Point b, Point c, Point d)
132 {
133     set <Point> ans;
134     if (eq((b - a) ^ (d - c), 0))
135     {
136         if (Point_on_segment(a, c, d)) ans.insert(a);
137         if (Point_on_segment(b, c, d)) ans.insert(b);
138         if (Point_on_segment(c, a, b)) ans.insert(c);
139         if (Point_on_segment(d, a, b)) ans.insert(d);
140     }
141     else
142     {
143         line ab = get_line(a, b), cd = get_line(c, d);
144         Point p = intersection_line_line(ab, cd);
145         if (Point_on_segment(p, a, b) && Point_on_segment(p, c, d)) ans.insert(p);
146     }
147     return ans;
148 }
149 double dist_seg_seg(Point a, Point b, Point c, Point d)
150 {
151     line ab = get_line(a, b);
152     line cd = get_line(c, d);
153     if (parallel(ab, cd))
154         return min({dist_Point_segment(c, a, b), dist_Point_segment(d, a, b),
155                    dist_Point_segment(a, c, d), dist_Point_segment(b, c, d)});
156     Point p = intersection_line_line(ab, cd);
157     if (Point_on_segment(p, a, b) && Point_on_segment(p, c, d))
158         return 0;
159     return min({dist_Point_segment(c, a, b), dist_Point_segment(d, a, b),
160                dist_Point_segment(a, c, d), dist_Point_segment(b, c, d)});
161 }
162 double dist_seg_ray(Point a, Point b, Point c, Point d)
163 {
164     line ab = get_line(a, b);
165     line cd = get_line(c, d);
166     if (parallel(ab, cd))
167         return min({dist_Point_ray(a, c, d), dist_Point_ray(b, c, d), dist_Point_segment(c, a, b)});
168     Point p = intersection_line_line(ab, cd);
169     if (Point_on_segment(p, a, b) && Point_on_ray(p, c, d))
170         return 0;
171     return min({dist_Point_ray(a, c, d), dist_Point_ray(b, c, d), dist_Point_segment(c, a, b)});
172 }
173 double dist_seg_line(Point a, Point b, Point c, Point d)
174 {
175     line ab = get_line(a, b);
176     line cd = get_line(c, d);
177     if (parallel(ab, cd))
178         return min(dist_Point_line(a, c, d), dist_Point_line(b, c, d));
179     Point p = intersection_line_line(ab, cd);
180     if (Point_on_segment(p, a, b) && Point_on_line(p, c, d))
181         return 0;
182     return min(dist_Point_line(a, c, d), dist_Point_line(b, c, d));
183 }
184 double dist_ray_ray(Point a, Point b, Point c, Point d)
185 {
186     line ab = get_line(a, b);
187     line cd = get_line(c, d);
188     if (parallel(ab, cd))
189         return min(dist_Point_ray(a, c, d), dist_Point_ray(c, a, b));
190     Point p = intersection_line_line(ab, cd);
191     if (Point_on_ray(p, a, b) && Point_on_ray(p, c, d))
192         return 0;
193     return min(dist_Point_ray(a, c, d), dist_Point_ray(c, a, b));
194 }
195 double dist_ray_line(Point a, Point b, Point c, Point d)
196 {
197     line ab = get_line(a, b);
198     line cd = get_line(c, d);
199     if (parallel(ab, cd))
200         return dist_Point_line(a, c, d);
201     Point p = intersection_line_line(ab, cd);
202     if (Point_on_ray(p, a, b) && Point_on_line(p, c, d))
203         return 0;
204     return dist_Point_line(a, c, d);
205 }

```

```

206 Point projection(line l, Point p)
207 {
208     Point normal = { l.a, l.b };
209     double d = dist_Point_line2(p, l);
210     normal = normal / normal.len() * d;
211     Point base = p + normal;
212     if (!eq(l.a*base.x + l.b*base.y + l.c, 0))
213         base = base - normal * 2;
214     return base;
215 }
216 pair<int, int> tangent_polygon(Point a, vector<Point> &p) {
217     int p1 = 0, p2 = 0, n = sz(p);
218     for (int i = 20; i >= 0; --i) {
219         {
220             int nxt = (p1 + (1 << i)) % n, prev = ((p1 - (1 << i)) % n + n) % n;
221             if (((p[p1] - a) ^ (p[nxt] - a)) > 0) {
222                 p1 = nxt;
223             }
224             if (((p[p1] - a) ^ (p[prev] - a)) > 0) {
225                 p1 = prev;
226             }
227         }
228         {
229             int nxt = (p2 + (1 << i)) % n, prev = ((p2 - (1 << i)) % n + n) % n;
230             if (((p[p2] - a) ^ (p[nxt] - a)) < 0) {
231                 p2 = nxt;
232             }
233             if (((p[p2] - a) ^ (p[prev] - a)) < 0) {
234                 p2 = prev;
235             }
236         }
237     }
238     return {p1, p2};
239 }
240 vector<Point> tangent_circle(Point o, double r, Point m) {
241     if (r > dist_Point_Point(o, m)) {
242         return {};
243     }
244     if (eq(r, dist_Point_Point(o, m))) {
245         return { m };
246     }
247     double d = r*r / dist_Point_Point(o, m);
248     Point om = m-o;
249     Point oh = om / om.len() * d;
250     Point h = o + oh;
251     line l = get_line(o, m);
252     Point normal = { l.a, l.b };
253     d = sqrtl(r*r - oh.len()*oh.len());
254     normal = normal / normal.len() * d;
255     return { h+normal, h-normal };
256 }
257 vector<Point> intersection_circle_circle(Point o1, double r1, Point o2, double r2) {
258     double d = dist_Point_Point(o1, o2);
259     if (o1 == o2 && eq(r1, r2)) {
260         return {};
261     }
262     if (d > r1+r2 || d < abs(r1-r2)) {
263         return {};
264     }
265     if (eq(d, r1+r2)) {
266         Point v = o2 - o1;
267         v = v / v.len() * r1;
268         return { o1 + v };
269     }
270     if (eq(d, abs(r1-r2))) {
271         if (r1 > r2) {
272             swap(r1, r2);
273             swap(o1, o2);
274         }
275         Point v = o1-o2;
276         v = v / v.len() * r2;
277         return { o2 + v };
278     }
279     double o1h_len = (d*d + r1*r1 - r2*r2) / (2*d);
280     Point v = o2-o1;
281     Point o1h = v / v.len() * o1h_len;
282     Point h = o1 + o1h;
283     line l = get_line(o1, o2);
284     Point normal = { l.a, l.b };
285     double dist = dist_Point_Point(o1, h);
286     d = sqrtl(r1*r1 - dist*dist);
287     normal = normal / normal.len() * d;
288     return { h+normal, h-normal };
289 }
290 void reorder(vector<Point> &p) {
291     int mn = 0;
292     for (int i = 1; i < sz(p); ++i) {
293         if (p[i] < p[mn]) {
294             mn = i;
295         }
296     }
297     rotate(p.begin(), p.begin() + mn, p.end());
298 }
299 vector<Point> sum_of_polygons(vector<Point> a, vector<Point> b) {
300     vector<Point> sum;
301     reorder(a), reorder(b);

```

```

302     a.push_back(a[0]), b.push_back(b[0]);
303     a.push_back(a[1]), b.push_back(b[1]);
304     int i = 0, j = 0;
305     while (i < sz(a) - 2 || j < sz(b) - 2) {
306         sum.push_back(a[i] + b[j]);
307         auto cross = ((a[i + 1] - a[i]) ^ (b[j + 1] - b[j]));
308         if ((cross > 0 || eq(cross, 0)) && i < sz(a) - 2) {
309             ++i;
310         }
311         if ((cross < 0 || eq(cross, 0)) && j < sz(b) - 2) {
312             ++j;
313         }
314     }
315     return sum;
316 }

```

Касотмный хэш

```

1 struct custom_hash {
2     static uint64_t splitmix64(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8     size_t operator()(uint64_t x) const {
9         static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();
10        return splitmix64(x + FIXED_RANDOM);
11    }
12 };
13 unordered_map<ll, ll, custom_hash>

```

Манакер

```

1 vector<int> d1(n);
2 int l=0, r=-1;
3 for (int i=0; i<n; ++i) {
4     int k = i>r ? 1 : min(d1[l+r-i], r-i+1);
5     while (i+k < n && i-k >= 0 && s[i+k] == s[i-k]) ++k;
6     d1[i] = k;
7     if (i+k-1 > r)
8         l = i-k+1, r = i+k-1;
9 }
10 vector<int> d2(n);
11 l=0, r=-1;
12 for (int i=0; i<n; ++i) {
13     int k = i>r ? 0 : min(d2[l+r-i+1], r-i+1);
14     while (i+k < n && i-k-1 >= 0 && s[i+k] == s[i-k-1]) ++k;
15     d2[i] = k;
16     if (i+k-1 > r)
17         l = i-k, r = i+k-1;
18 }

```

Прагмы

```

1 #pragma GCC optimize("Ofast")
2 #pragma GCC target("sse,sse2,sse3,ssse3,sse4,popcnt,abm,mmx,tune=native")
3 #pragma GCC optimize("unroll-loops")

```

Расстояние Левенштейна

```

1 int levenshtein(const string& A, const string& B) {
2     int n = (int)A.size(), m = (int)B.size();
3     vector<int> prev(m+1), cur(m+1);
4     iota(prev.begin(), prev.end(), 0); // dp[0][j] = j
5     for (int i = 1; i <= n; ++i) {
6         cur[0] = i; // dp[i][0] = i
7         for (int j = 1; j <= m; ++j) {
8             int cost = (A[i-1] == B[j-1]) ? 0 : 1;
9             cur[j] = min({
10                 prev[j] + 1, // delete A[i-1]
11                 cur[j-1] + 1, // insert B[j-1]
12                 prev[j-1] + cost // replace/match
13             });
14         }
15         swap(prev, cur);
16     }
17     return prev[m];
18 }

```

КМП (поиск подстроки в строке)

```

1 vector<int> build_pi(string_view p) {
2     int m = (int)p.size();
3     vector<int> pi(m, 0);
4     for (int i = 1; i < m; ++i) {
5         int j = pi[i-1];
6         while (j > 0 && p[i] != p[j]) j = pi[j-1];
7         if (p[i] == p[j]) ++j;
8         pi[i] = j;
9     }
10    return pi;

```

```

11 }
12 vector<int> kmp_search(string_view s, string_view p) { // find p in s
13     vector<int> res;
14     if (p.empty()) {
15         res.reserve(s.size() + 1);
16         for (int i = 0; i <= (int)s.size(); ++i) res.push_back(i);
17         return res;
18     }
19     vector<int> pi = build_pi(p);
20     int n = (int)s.size(), m = (int)p.size();
21     int j = 0;
22     for (int i = 0; i < n; ++i) {
23         while (j > 0 && s[i] != p[j]) j = pi[j - 1];
24         if (s[i] == p[j]) ++j;
25         if (j == m) {
26             res.push_back(i - m + 1);
27             j = pi[j - 1];
28         }
29     }
30     return res;
31 }

```

Ахо-корасик (с суффиксными переходами)

```

1  const int k = 26;
2
3  struct vertex {
4      vertex* to[k] = {0};
5      vertex* go[k] = {0};
6      vertex* link = nullptr;
7      vertex* out_link = nullptr;
8      vertex* p = nullptr;
9      int pch = -1;
10     bool term = false;
11     vector<int> out;
12     vertex(int _pch, vertex* _p) : p(_p), pch(_pch) {}
13 };
14
15 vertex* root = new vertex(-1, nullptr);
16
17 int cid(char c) {
18     return c - 'a';
19 }
20
21 void add_string(const string& s, int id) {
22     vertex* v = root;
23     for (char ch : s) {
24         int c = cid(ch);
25         if (c < 0 || c >= k) {
26             continue;
27         }
28         if (!v->to[c]) {
29             v->to[c] = new vertex(c, v);
30         }
31         v = v->to[c];
32     }
33     v->term = true;
34     v->out.push_back(id);
35 }
36
37 vertex* go(vertex* v, int c);
38
39 vertex* link(vertex* v) {
40     if (v->link) {
41         return v->link;
42     }
43     if (v == root || v->p == root) {
44         return v->link = root;
45     }
46     return v->link = go(link(v->p), v->pch);
47 }
48
49 vertex* go(vertex* v, int c) {
50     if (v->go[c]) {
51         return v->go[c];
52     }
53     if (v->to[c]) {
54         return v->go[c] = v->to[c];
55     }
56     if (v == root) {
57         return v->go[c] = root;
58     }
59     return v->go[c] = go(link(v), c);
60 }
61
62 vertex* out_link(vertex* v) {
63     if (v->out_link) {
64         return v->out_link;
65     }
66     vertex* u = link(v);
67     if (u == v) {
68         return v->out_link = nullptr;
69     }
70     if (!u) {
71         return v->out_link = nullptr;

```

```

72     }
73     if (u->term) {
74         return v->out_link = u;
75     }
76     return v->out_link = out_link(u);
77 }
78
79 template<class f>
80 void find_all(const string& text, f on_match) {
81     vertex* v = root;
82     for (int i = 0; i < (int)text.size(); ++i) {
83         int c = cid(text[i]);
84         if (c < 0 || c >= k) {
85             v = root;
86             continue;
87         }
88         v = go(v, c);
89         if (v->term) {
90             for (int id : v->out) {
91                 on_match(i, id);
92             }
93         }
94         for (vertex* u = out_link(v); u; u = out_link(u)) {
95             for (int id : u->out) {
96                 on_match(i, id);
97             }
98         }
99     }
100 }

```

Ахо-корасик (без суффиксных переходов)

```

1 struct node {
2     int nxt[26];
3     int term = 0;
4     int p = -1;
5     int pch;
6     int link = -1, super_link = -1;
7
8     node(int p = -1, int ch = -1) : p(p), pch(ch) {
9         fill(begin(nxt), end(nxt), 0);
10    }
11 };
12
13 vector<node> trie(1);
14
15 void add(string &s) {
16     int v = 0;
17     for (char ch: s) {
18         int c = ch - 'a';
19         if (trie[v].nxt[c] == 0) {
20             trie[v].nxt[c] = sz(trie);
21             trie.emplace_back(v, c);
22         }
23         v = trie[v].nxt[c];
24     }
25     trie[v].term = 1;
26 }
27
28 void bfs() {
29     queue<int> q;
30     q.push(0);
31     while (!q.empty()) {
32         int v = q.front();
33         q.pop();
34         for (int c: trie[v].nxt) {
35             if (c != 0)
36                 q.push(c);
37         }
38         if (v == 0) {
39             trie[v].link = trie[v].super_link = 0;
40             continue;
41         }
42         if (trie[v].p == 0) {
43             trie[v].link = trie[v].super_link = 0;
44         } else {
45             trie[v].link = trie[trie[trie[v].p].link].nxt[trie[v].pch];
46             trie[v].super_link = (trie[trie[v].link].term ? trie[v].link : trie[trie[v].link].
super_link);
47         }
48         for (int c = 0; c < 26; ++c) {
49             if (trie[v].nxt[c] == 0)
50                 trie[v].nxt[c] = trie[trie[v].link].nxt[c];
51         }
52     }
53 }

```

Суффиксный автомат

```

1 const int K = 26;
2 struct sam {
3     struct state {
4         int link;
5         int len;
6         int next[K];

```

```

7         state() {
8             link = -1;
9             len = 0;
10            for (int i = 0; i < K; ++i) {
11                next[i] = -1;
12            }
13        }
14    };
15    vector<state> st;
16    int last;
17    sam() {
18        st.push_back(state());
19        last = 0;
20    }
21    int cid(char c) {
22        if ('A' <= c && c <= 'Z') {
23            return c - 'A';
24        }
25        return c - 'a';
26    }
27    void extend_char(int c) {
28        int cur = st.size();
29        st.push_back(state());
30        st[cur].len = st[last].len + 1;
31        int p = last;
32        while (p != -1 && st[p].next[c] == -1) {
33            st[p].next[c] = cur;
34            p = st[p].link;
35        }
36        if (p == -1) {
37            st[cur].link = 0;
38        } else {
39            int q = st[p].next[c];
40            if (st[p].len + 1 == st[q].len) {
41                st[cur].link = q;
42            } else {
43                int clone = st.size();
44                st.push_back(st[q]);
45                st[clone].len = st[p].len + 1;
46                while (p != -1 && st[p].next[c] == q) {
47                    st[p].next[c] = clone;
48                    p = st[p].link;
49                }
50                st[q].link = clone;
51                st[cur].link = clone;
52            }
53        }
54        last = cur;
55    }
56    void extend_string(const string& s) {
57        for (char ch : s) {
58            int c = cid(ch);
59            if (c < 0 || c >= K) {
60                last = 0;
61                continue;
62            }
63            extend_char(c);
64        }
65    }
66    int count_distinct_substrings() {
67        int ans = 0;
68        for (int v = 1; v < (int)st.size(); ++v) {
69            ans += st[v].len - st[st[v].link].len;
70        }
71        return ans;
72    }
73    bool contains(const string& t) {
74        int u = 0;
75        for (char ch : t) {
76            int c = cid(ch);
77            if (st[u].next[c] == -1) {
78                return false;
79            }
80            u = st[u].next[c];
81        }
82        return true;
83    }
84 };

```

Римские числа

```

1 int RomanToDecimal(const std::string& roman) {
2     std::unordered_map<char, int> values = {
3         {'I', 1}, {'V', 5}, {'X', 10}, {'L', 50},
4         {'C', 100}, {'D', 500}, {'M', 1000}
5     };
6     int result = 0;
7     int n = roman.size();
8     for (int i = 0; i < n; ++i) {
9         int value = values[roman[i]];
10        if (i + 1 < n && values[roman[i + 1]] > value) {
11            result -= value;
12        } else {
13            result += value;
14        }
15    }

```

```

16     return result;
17 }
18
19 string DecimalToRoman(int num) {
20     if (num <= 0 || num > 3999)
21         throw invalid_argument("num must be in 1..3999");
22
23     static const pair<int, const char*> map[] = {
24         {1000, "M"}, {900, "CM"}, {500, "D"}, {400, "CD"},
25         {100, "C"}, {90, "XC"}, {50, "L"}, {40, "XL"},
26         {10, "X"}, {9, "IX"}, {5, "V"}, {4, "IV"},
27         {1, "I"}
28     };
29
30     string out;
31     for (auto [val, sym] : map) {
32         while (num >= val) {
33             out += sym;
34             num -= val;
35         }
36     }
37     return out;
38 }

```

Венгерский алгоритм

Дана матрица a размера $n \times n$. Требуется найти такую перестановку p длины n , что величина $\sum a[i][p[i]]$ — минимальна.

Решение написано в 1-индексации, чтобы решать задачу максимизации нужно умножить $a[i][j]$ на -1.

```

1 vector<int> u(n + 1), v(m + 1), p(m + 1), way(m + 1);
2 for (int i = 1; i <= n; ++i) {
3     p[0] = i;
4     int j0 = 0;
5     vector<int> minv(m + 1, INF);
6     vector<char> used(m + 1, false);
7     do {
8         used[j0] = true;
9         int i0 = p[j0], delta = INF, j1;
10        for (int j = 1; j <= m; ++j)
11            if (!used[j]) {
12                int cur = a[i0][j] - u[i0] - v[j];
13                if (cur < minv[j])
14                    minv[j] = cur, way[j] = j0;
15                if (minv[j] < delta)
16                    delta = minv[j], j1 = j;
17            }
18        for (int j = 0; j <= m; ++j)
19            if (used[j])
20                u[p[j]] += delta, v[j] -= delta;
21        else
22            minv[j] -= delta;
23        j0 = j1;
24    } while (p[j0] != 0);
25    do {
26        int j1 = way[j0];
27        p[j0] = p[j1];
28        j0 = j1;
29    } while (j0);
30 }
31 vector<int> ans (n+1);
32 for (int j = 1; j <= m; ++j)
33     ans[p[j]] = j;

```

Эйлеров путь

```

1 vector<vector<pii>> g;
2 vector<int> fr, used, e;
3 void dfs(int v) {
4     while (fr[v] < sz(g[v])) {
5         auto &[to, id] = g[v][fr[v]];
6         ++fr[v];
7         if (used[id])
8             continue;
9         used[id] = true;
10        dfs(to);
11    }
12    e.eb(v);
13 }
14 void solve() {
15     int n, m;
16     vector<int> odd;
17     vector<pii> edges;
18     // INPUT
19     fr.resize(n);
20     g.resize(n);
21     for (auto &[u, v] : edges) {
22         g[u].emplace_back(v, m);
23         g[v].emplace_back(u, m++);
24     }
25     int v1 = 0, v2;
26     bool f = false;
27     if (odd.size() == 2) {

```

```

28     v1 = odd[0];
29     v2 = odd[1];
30     g[v1].emplace_back(v2, m);
31     g[v2].emplace_back(v1, m++);
32     odd.clear();
33     f = true;
34 }
35 if (!odd.empty()) {
36     cout << -1;
37     return;
38 }
39 used.resize(m);
40 dfs(v1);
41 if (f) {
42     for (int i = 0; i < m - 1; ++i) {
43         if (e[i] == v1 && e[i + 1] == v2 || e[i] == v2 && e[i + 1] == v1) {
44             vector<int> new_e(e.begin() + i + 1, e.end());
45             new_e.insert(new_e.end(), e.begin(), e.begin() + i);
46             e = new_e;
47             break;
48         }
49     }
50     --m;
51 }
52 // euler tour in e
53 }

```

Ли Чао с добавлением прямой на отрезке

```

1 struct Line {
2     ll a, b;
3     Line(ll a = 0, ll b = INF): a(a), b(b) {};
4     ll getY(ll x) {
5         return a * x + b;
6     }
7 };
8
9 struct Node {
10     Line L;
11     Node(Line L = Line()): L(L) {};
12 };
13
14 const int N = 2e5, MOD = 2e5;
15 int q;
16 ll p = 0;
17 Node t[N * 20];
18
19 void insert_line(int v, ll tl, ll tr, Line l) {
20     if (tl > tr) {
21         return;
22     }
23     ll tm = tl + (tr - tl) / 2;
24     Line l2 = t[v].L;
25     int l_lower = l.getY(tl) < l2.getY(tl);
26     int m_lower = l.getY(tm) < l2.getY(tm);
27     if (m_lower) {
28         swap(l, t[v].L);
29     }
30     if (tl == tr) {
31         return;
32     }
33     if (l_lower ^ m_lower) {
34         insert_line(v * 2, tl, tm, l);
35     } else {
36         insert_line(v * 2 + 1, tm + 1, tr, l);
37     }
38 }
39
40 void update(int v, ll tl, ll tr, ll ql, ll qr, Line l) {
41     if (tr < ql || qr < tl) {
42         return;
43     }
44     if (ql <= tl && tr <= qr) {
45         insert_line(v, tl, tr, l);
46         return;
47     }
48     ll tm = tl + (tr - tl) / 2;
49     update(v * 2, tl, tm, ql, qr, l);
50     update(v * 2 + 1, tm + 1, tr, ql, qr, l);
51 }
52
53 ll get_min(int v, ll tl, ll tr, ll x) {
54     if (tl > tr) {
55         return INF;
56     }
57     ll res = t[v].L.getY(x), tm = tl + (tr - tl) / 2;
58     if (tl == tr) {
59         return res;
60     }
61     if (x <= tm) {
62         res = min(res, get_min(v * 2, tl, tm, x));
63         return res;
64     } else {
65         res = min(res, get_min(v * 2 + 1, tm + 1, tr, x));
66         return res;
67     }
68 }

```

```

68 }
69
70 void solve() {
71     cin >> q;
72     while (q--) {
73         int tp; cin >> tp;
74         if (tp == 1) {
75             ll a, b, l, r;
76             cin >> a >> b >> l >> r;
77             a += p, b += p;
78             update(1, -N, N, l, r, Line(a, b));
79         } else {
80             ll x; cin >> x;
81             p = get_min(1, -N, N, x);
82             p = (p == INF ? -1 : p);
83             cout << p << '\n';
84             p = (p % MOD + MOD) % MOD;
85         }
86     }
87 }

```

2-SAT

```

1  const int N = 100010;
2  int n, m;
3  vector<int> g[N * 2], rg[N * 2];
4  int used[N * 2];
5  vector<int> ts;
6
7  void dfs(int v) {
8      used[v] = 1;
9      for (int i: g[v]) {
10         if (!used[i]) dfs(i);
11     }
12     ts.push_back(v);
13 }
14
15 void dfs2(int v, int c) {
16     used[v] = c;
17     for (int i: rg[v]) {
18         if (!used[i])
19             dfs2(i, c);
20     }
21 }
22
23 void solve() {
24     while (cin >> n >> m) {
25         for (int i = 0; i < 2 * n; ++i)
26             g[i] = rg[i] = {}, used[i] = 0;
27         ts = {};
28         for (int i = 1; i <= m; ++i) {
29             int i1, e1, i2, e2;
30             cin >> i1 >> e1 >> i2 >> e2;
31             g[i1 * 2 + (e1 ^ 1)].push_back(i2 * 2 + e2);
32             rg[i2 * 2 + e2].push_back(i1 * 2 + (e1 ^ 1));
33             g[i2 * 2 + (e2 ^ 1)].push_back(i1 * 2 + e1);
34             rg[i1 * 2 + e1].push_back(i2 * 2 + (e2 ^ 1));
35         }
36         for (int i = 0; i < 2 * n; ++i) {
37             if (!used[i]) dfs(i);
38         }
39         for (int i = 0; i < 2 * n; ++i) {
40             used[i] = 0;
41         }
42         reverse(all(ts));
43         for (int i = 0, j = 1; i < 2 * n; ++i) {
44             if (!used[ts[i]]) dfs2(ts[i], j++);
45         }
46         bool possible = true;
47         for (int i = 0; i < n; ++i) {
48             if (used[2 * i] == used[2 * i + 1]) {
49                 possible = false;
50                 break;
51             }
52         }
53         for (int i = 0; i < n; ++i)
54             cout << (used[i * 2] < used[i * 2 + 1] ? 1 : 0);
55         cout << '\n';
56     }
57 }

```

HLD

```

1  const int N = 1e5 + 1;
2  vector<int> g[N];
3  int sz[N], p[N], tin[N], tout[N], head[N], t[N * 10];
4  int timer = 1, n;
5
6  void sizes(int v, int anc) {
7      sz[v] = 1;
8      for (int &i: g[v]) {
9          if (i == anc)
10             continue;
11         p[i] = v;
12         sizes(i, v);

```

```

13         sz[v] += sz[i];
14         if (sz[i] > sz[g[v][0]])
15             swap(i, g[v][0]);
16     }
17 }
18
19 void hld(int v, int anc) {
20     tin[v] = timer++;
21     for (int i: g[v]) {
22         if (i == anc)
23             continue;
24         head[i] = (i == g[v][0] ? head[v] : i);
25         hld(i, v);
26     }
27     tout[v] = timer;
28 }
29
30 int upper(int a, int b) {
31     return tin[a] <= tin[b] && tout[a] >= tout[b];
32 }
33
34 int get(int v, int tl, int tr, int ql, int qr) {
35     if (qr < tl || tr < ql)
36         return -inf;
37     if (ql <= tl && tr <= qr)
38         return t[v];
39     int m = (tl + tr) >> 1;
40     return max(get(v * 2, tl, m, ql, qr), get(v * 2 + 1, m + 1, tr, ql, qr));
41 }
42
43 void add(int v, int tl, int tr, int pos, int x) {
44     if (tl == tr) {
45         t[v] += x;
46         return;
47     }
48     int m = (tl + tr) >> 1;
49     if (pos <= m) add(v * 2, tl, m, pos, x);
50     else add(v * 2 + 1, m + 1, tr, pos, x);
51     t[v] = max(t[v * 2], t[v * 2 + 1]);
52 }
53
54 void up(int &a, int &b, int &res) {
55     while (!upper(head[a], b)) {
56         res = max(res, get(1, 1, timer, tin[head[a]], tin[a]));
57         a = p[head[a]];
58     }
59 }
60
61 int get_max(int a, int b) {
62     int res = -inf;
63     up(a, b, res);
64     up(b, a, res);
65     if (!upper(a, b))
66         swap(a, b);
67     res = max(res, get(1, 1, timer, tin[a], tin[b]));
68     return res;
69 }
70 //TODO p[root] = head[root] = root;

```

Центроидная декомпозиция

```

1  const int N = 100010, LOG = 20;
2  vector<int> g[N];
3  int cent[N], _size[N], pr[N][LOG];
4  vector<int> a;
5  // TODO fill(cent, cent + N, -1)
6  int dfs(int v, int p) {
7      if (cent[v] != -1)
8          return 0;
9      _size[v] = 1;
10     for (int i: g[v]) {
11         if (i == p)
12             continue;
13         _size[v] += dfs(i, v);
14     }
15     a.push_back(v);
16     return _size[v];
17 }
18
19 void dfs2(int v, int p, int c, int level) {
20     if (cent[v] != -1)
21         return;
22     pr[v][level] = c;
23     for (int i: g[v]) {
24         if (i == p)
25             continue;
26         dfs2(i, v, c, level);
27     }
28 }
29
30 void find_centroid(int v, int level) {
31     a.clear();
32     int s = dfs(v, -1);
33     for (int i: a) {
34         if (_size[i] > s / 2) {
35             v = i;

```

```

36         break;
37     }
38 }
39 dfs2(v, -1, v, level);
40 cent[v] = level;
41 for (int i: g[v]) {
42     if (cent[i] != -1)
43         find_centroid(i, level + 1);
44 }
45 }

```

Декартач по неявному ключу

```

1 mt19937 Rand(chrono::steady_clock::now().time_since_epoch().count());
2 struct Node {
3     Node *l, *r;
4     int x;
5     int g;
6     int sz;
7     int y;
8     Node(int x) : x(x), g(x), sz(1) {
9         y = (int)Rand();
10        l = r = nullptr;
11    }
12 };
13
14 Node *treap = nullptr;
15
16 int get_sz(Node *t) {
17     return t ? t->sz : 0;
18 }
19
20 int get_g(Node *t) {
21     return t ? t->g : 0;
22 }
23
24 void recalc(Node *t) {
25     if (!t) return;
26     t->sz = 1 + get_sz(t->l) + get_sz(t->r);
27     t->g = t->x;
28     t->g = gcd(t->g, get_g(t->l));
29     t->g = gcd(t->g, get_g(t->r));
30 }
31
32 Node *merge(Node *t1, Node *t2) {
33     if (!t1) return t2;
34     if (!t2) return t1;
35     if (t1->y >= t2->y) {
36         t1->r = merge(t1->r, t2);
37         recalc(t1);
38         return t1;
39     } else {
40         t2->l = merge(t1, t2->l);
41         recalc(t2);
42         return t2;
43     }
44 }
45
46 pair<Node*, Node*> split(Node *t, int k) {
47     if (!t) {
48         return {nullptr, nullptr};
49     }
50     int left_sz = get_sz(t->l);
51     if (left_sz >= k) {
52         auto [a, b] = split(t->l, k);
53         t->l = b;
54         recalc(t);
55         return {a, t};
56     } else {
57         auto [a, b] = split(t->r, k - left_sz - 1);
58         t->r = a;
59         recalc(t);
60         return {t, b};
61     }
62 }
63
64 int get_gcd(int l, int r) {
65     auto [a, bc] = split(treap, l);
66     auto [b, c] = split(bc, r - l + 1);
67     int ans = get_g(b);
68     treap = merge(a, merge(b, c));
69     return ans;
70 }
71
72 void update(int pos, int x) {
73     auto [a, bc] = split(treap, pos);
74     auto [b, c] = split(bc, 1);
75     delete b;
76     treap = merge(a, merge(new Node(x), c));
77 }
78
79 void insert(int pos, int x) {
80     auto [a, b] = split(treap, pos);
81     treap = merge(a, merge(new Node(x), b));
82 }
83

```

```
84 void erase(int pos) {  
85     auto [a, bc] = split(treap, pos);  
86     auto [b, c] = split(bc, 1);  
87     delete b;  
88     treap = merge(a, c);  
89 }
```